

Система мультифакторной аутентификации

Тимин Александр Сергеевич, гр. Б22-515

Научный руководитель:

Басыня Евгений Александрович, к.т.н, доцент, в.н.с

Кафедра криптологии и кибербезопасности
kaf42.mephi.ru



КАФЕДРА
КРИПТОЛОГИЯ И
КИБЕРБЕЗОПАСНОСТЬ

Актуальность

Запросы со стороны государства:

- 1) указ Президента РФ от 07.07.2011 № 899 «Об утверждении приоритетных направлений развития науки, технологий и техники в Российской Федерации и перечня критических технологий Российской Федерации»;
- 2) приказ ФСТЭК России № 21 от 18.02.2013 «Об утверждении Состава и содержания организационных и технических мер по защите персональных данных»: для высоких уровней защищённости персональных данных технически требуется многофакторная аутентификация (в т.ч. с использованием USB-токенов);
- 3) методический документ ФСТЭК России (утв. 12.04.2026): внешний удалённый доступ (RDP, Telnet, FTP) – только с MFA или сертификатами [1];

[1] ФСТЭК России переписала требования к защите данных в госсистемах и критической инфраструктуре // CISOCLUB [Электронный ресурс]. – URL: <https://cisoclub.ru/fstjek-rossii-perepisala-trebovanija-k-zashhite-dannyh-v-gossistemah-i-kriticheskoy-infrastrukture/> (дата обращения: 04.05.2026).

Актуальность

Запросы со стороны общества:

- 1) аналитический отчёт CISOCLUB: ~18% уязвимостей приходится на категорию A07 OWASP Top 10 (провалы идентификации и аутентификации, в т.ч. обход MFA) [2];
- 2) данные Microsoft: >99,9% скомпрометированных аккаунтов не используют MFA [3];
- 3) данные «Информзащиты»: в организациях без MFA успешны до 40% bruteforce-атак, при использовании MFA – не более 1% [4];

[2] Аналитический отчёт «Проведение анализа защищённости внешнего периметра (2022–2025 гг.)» // CISOCLUB [Электронный ресурс]. – URL: <https://cisoclub.ru/analiticheskij-otchet-2022-2025-podtverzhdennye-kriticheskie-ujazvimosti-na-perimetre-rossijskih-kompanij/> (дата обращения: 04.05.2026).

[3] Безопасность в организации: статистика многофакторной проверки подлинности // learn.microsoft.com [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/partner-center/security/security-at-your-organization> (дата обращения: 04.05.2026).

[4] MFA снижает эффективность bruteforce-атак до 1% // ITWeek [Электронный ресурс]. – URL: <https://www.itweek.ru/security/news-company/detail.php?ID=233607> (дата обращения: 04.05.2026).

Цель и задачи

Цель работы:

Обеспечение аутентичности пользователей информационных ресурсов вычислительной сети посредством мультифакторной аутентификации.

Задачи работы:

- 1) исследование предметной области;
- 2) моделирование предметной области;
- 3) проектирование системы мультифакторной аутентификации;
- 4) программная реализация спроектированной системы;
- 5) тестирование и рефакторинг реализованной системы.

Исследование предметной области

Фактор аутентификации – определенный вид аутентификатора.

Факторы аутентификации:

- 1) знание (пароль, PIN-код);
- 2) владение (физический ключ, смарт-карта, OTP-токен);
- 3) свойство (отпечаток пальца, рисунок радужки глаза, клавиатурный почерк).

Мультифакторная аутентификация (MFA) – комбинирование 2 и более факторов.

Исследование предметной области

Таблица 1 – Сравнение популярных MFA-систем от Яндекс и Google

Критерий сравнения	«Яндекс.Паспорт»	«Authenticator»
Комбинация факторов	2-3 или 1-2	2-
Направление аутентификации	Односторонняя	Односторонняя
Криптография аутентификатора	Хэширование	Хэширование
Встраиваемость	-	+
Кроссплатформенность	+	+

Обобщенная модель аутентификации

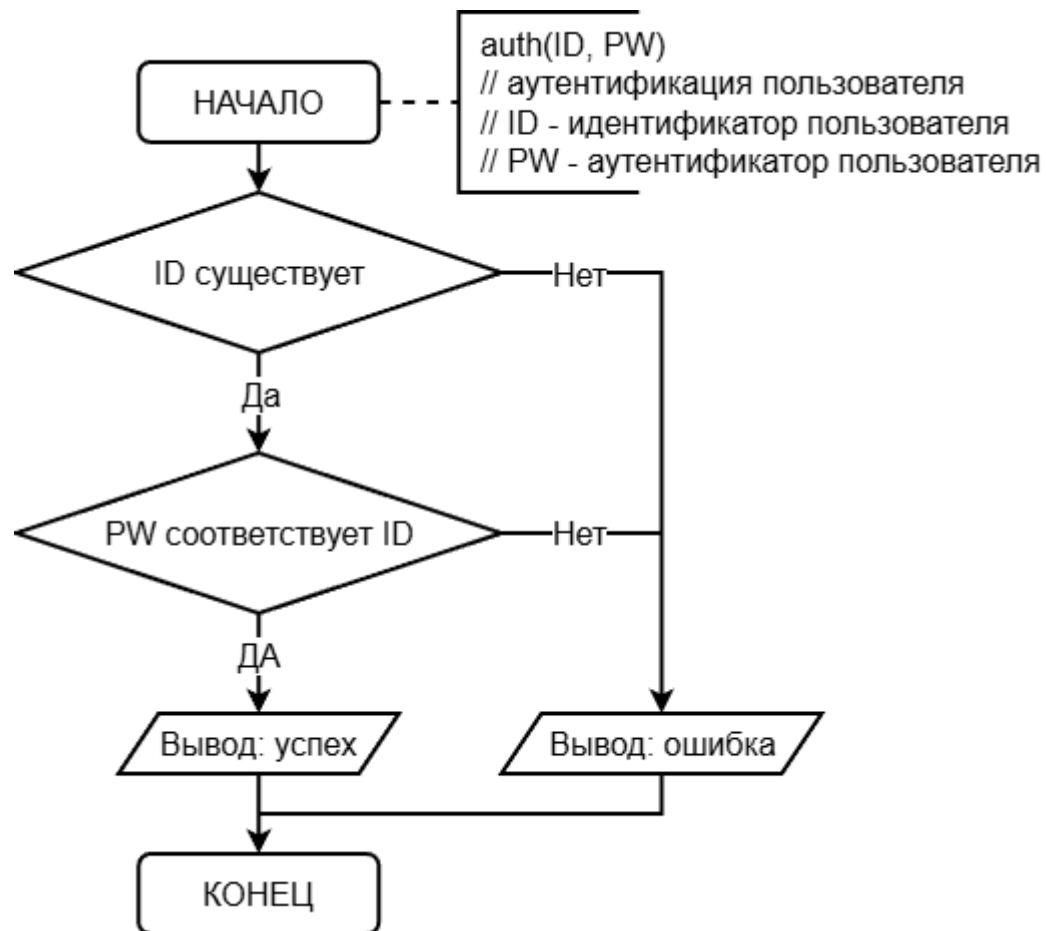


Рисунок 1 – блок-схема обобщенного алгоритма аутентификации



Рисунок 2 – UML-диаграмма последовательности обобщенного процесса аутентификации

Модель OTP-аутентификации

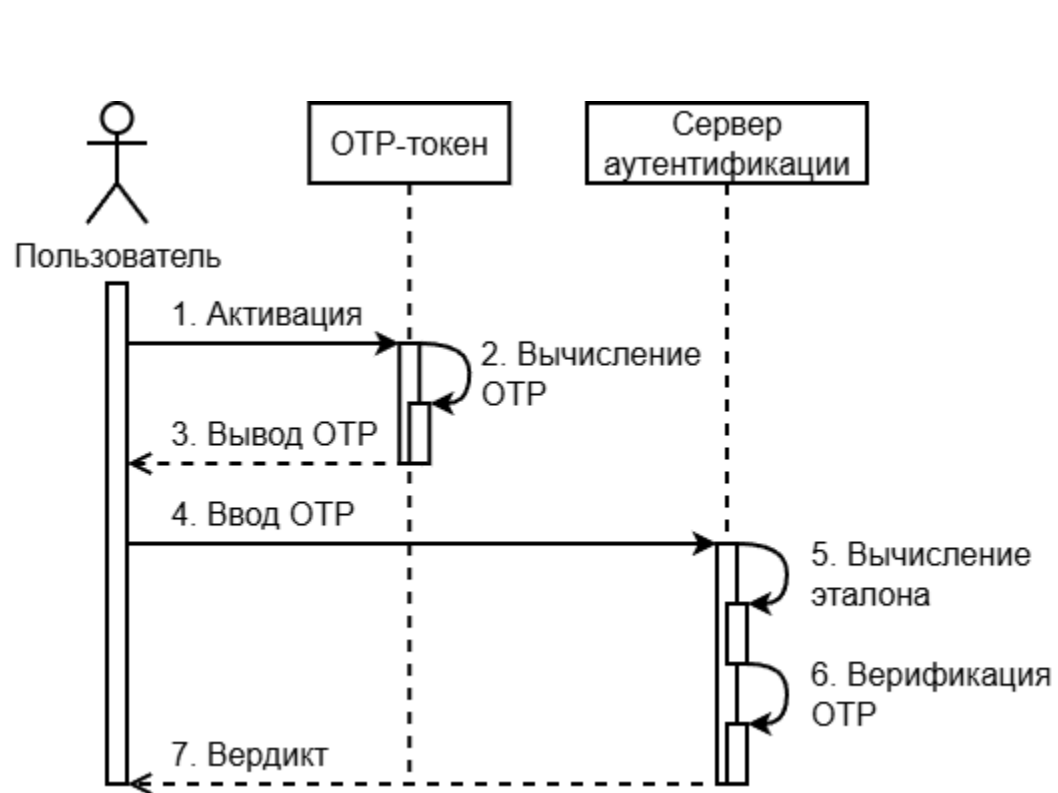


Рисунок 3 – UML-диаграмма последовательности синхронного процесса OTP-аутентификации

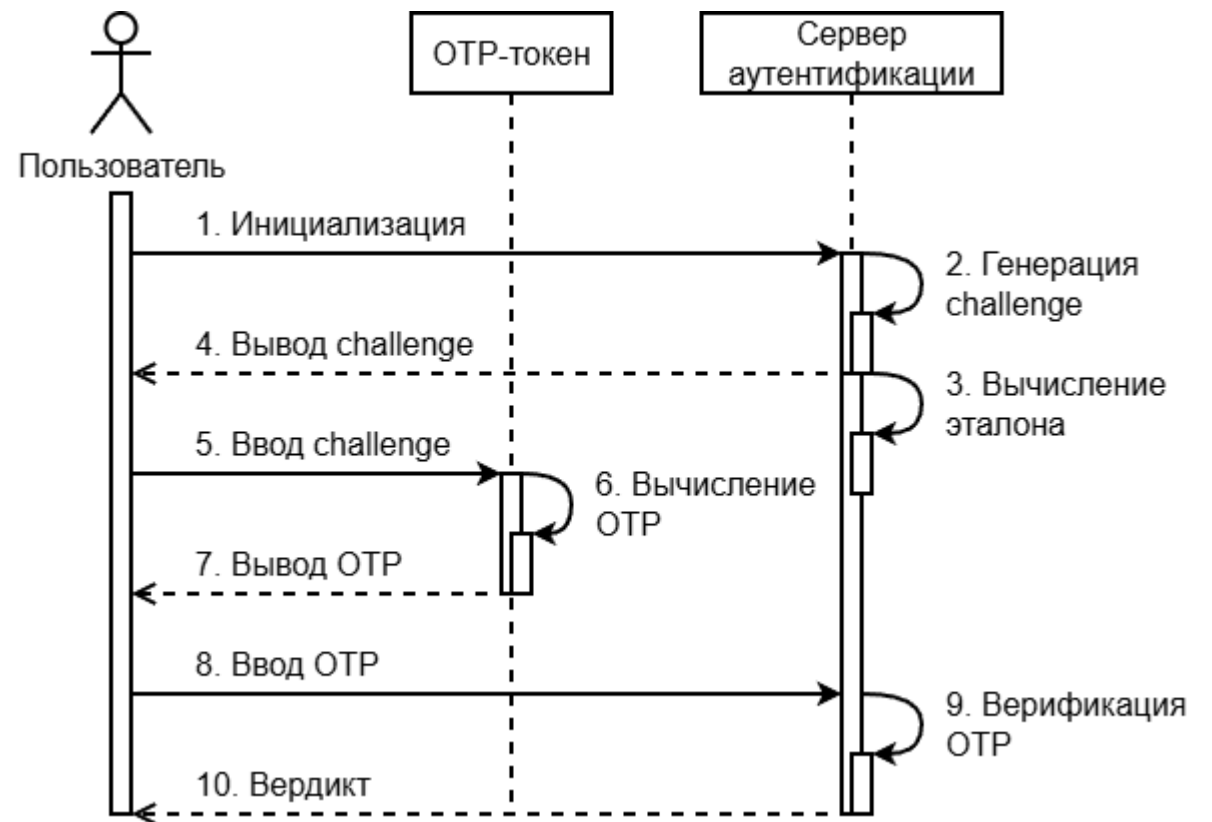


Рисунок 4 – UML-диаграмма последовательности асинхронного процесса OTP-аутентификации

Проектирование системы

Требования к реализации системы:

- 1) двусторонняя аутентификация;
- 2) встраиваемость системы;
- 3) шифрование аутентификаторов;
- 4) трехфакторная аутентификация;
- 5) аутентификация владельца OTP-токена;
- 6) аутентификация событию и по времени;
- 7) ограничение доступа при достижении лимита неудач.

Проектирование системы: КОМПОНЕНТЫ

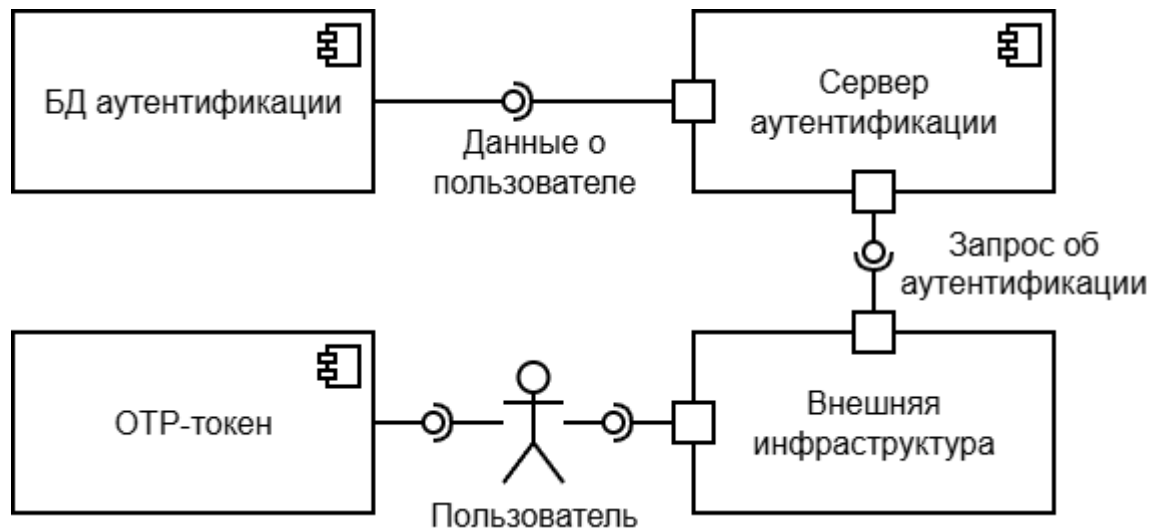


Рисунок 5 – UML-диаграмма компонентов

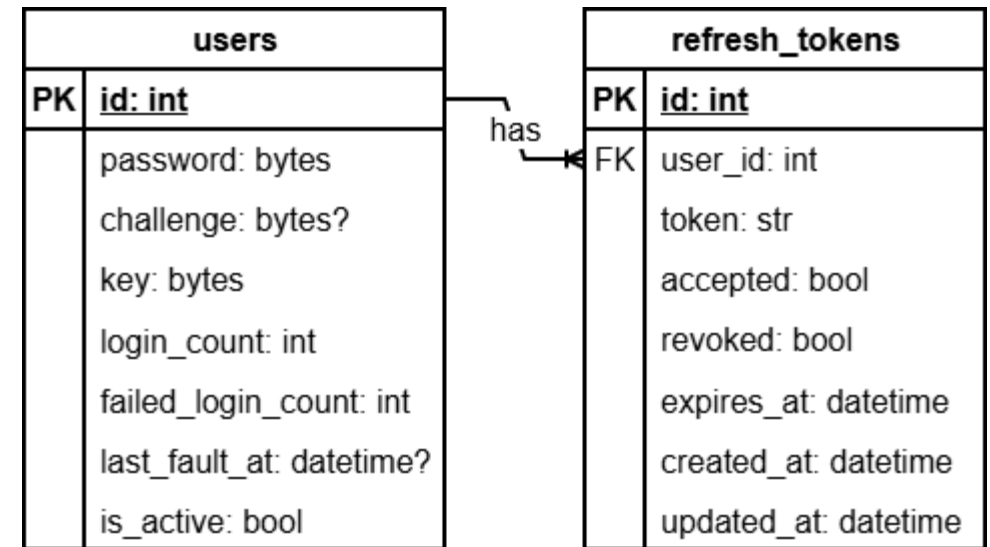


Рисунок 6 – ER-диаграмма БД аутентификации

Проектирование системы: процессы

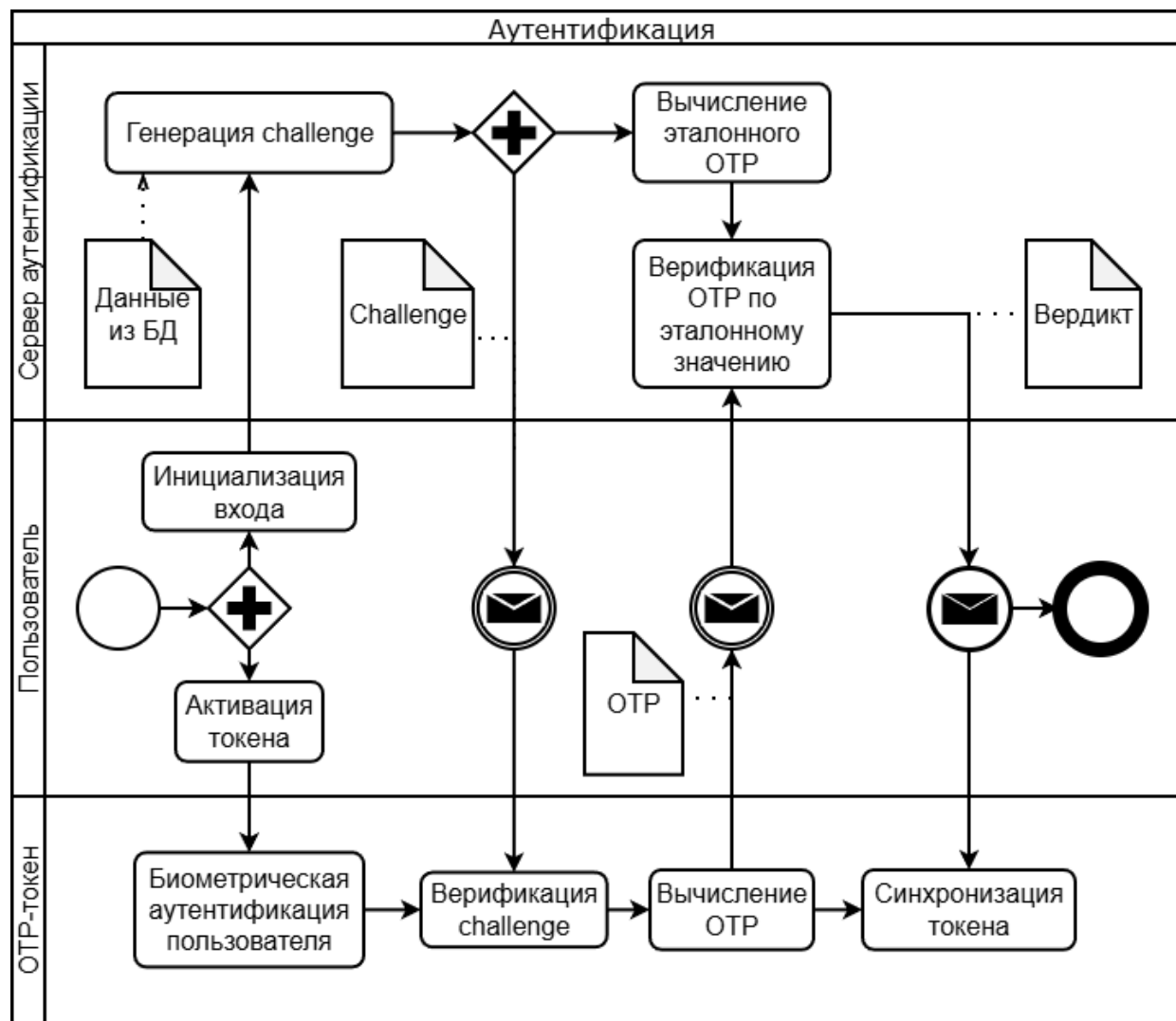


Рисунок 7 – BPMN-диаграмма сценария первичной аутентификации пользователя

Проектирование системы: ОТР-токен

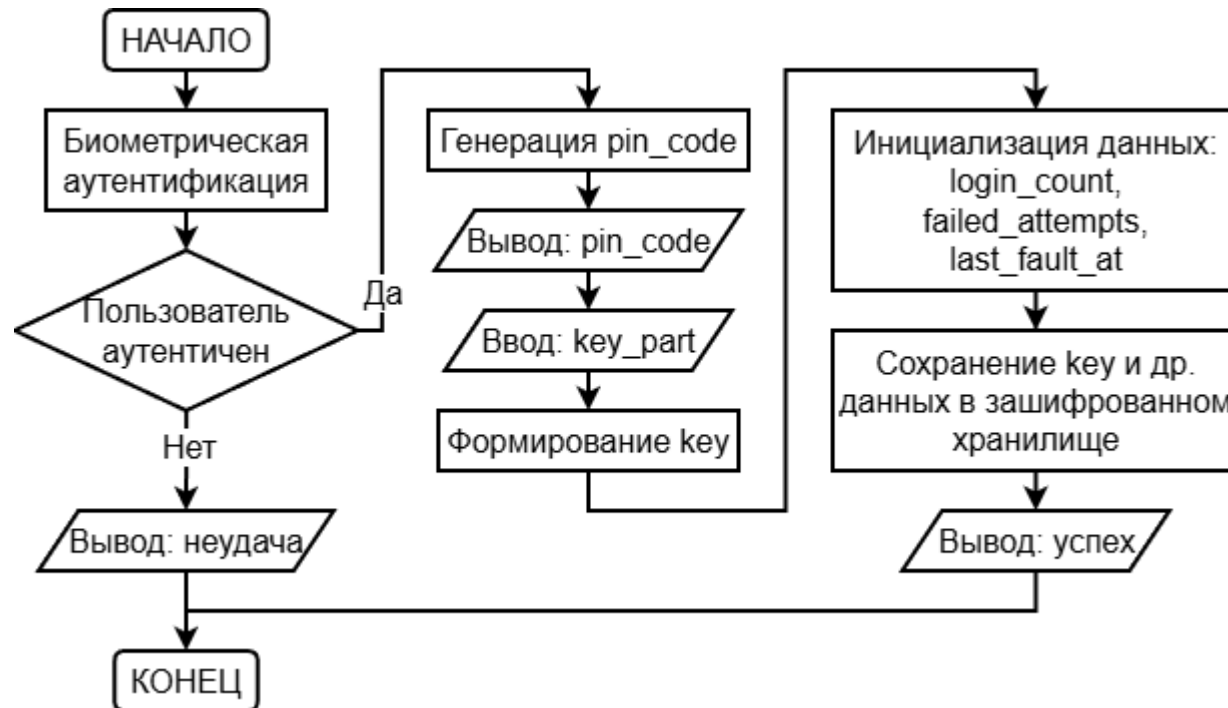


Рисунок 8 – блок-схема алгоритма регистрации ОТР-токена

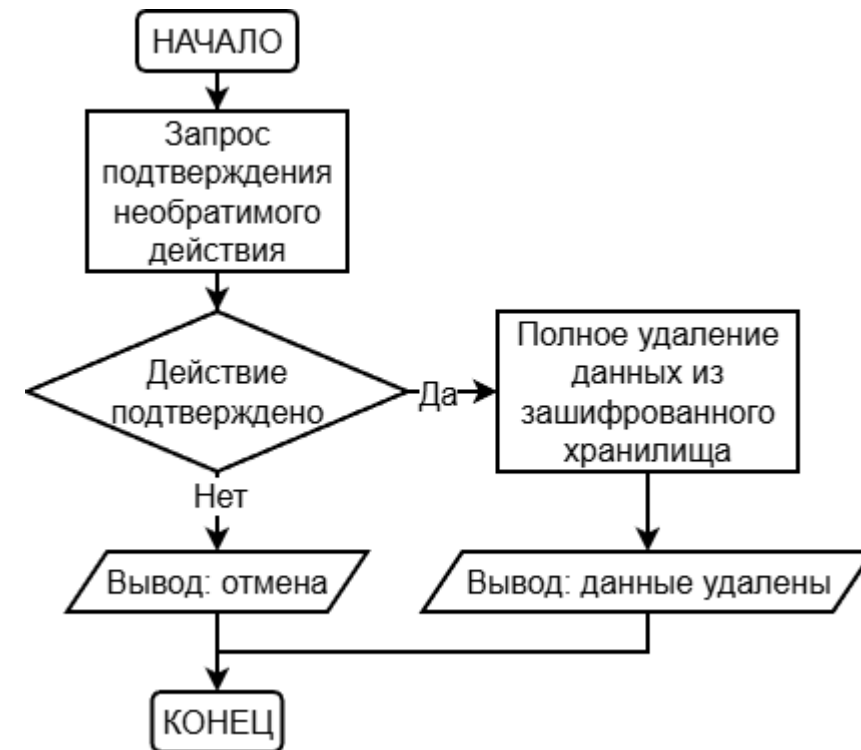


Рисунок 9 – блок-схема алгоритма сброса ОТР-токена

Проектирование системы: OTP-токен

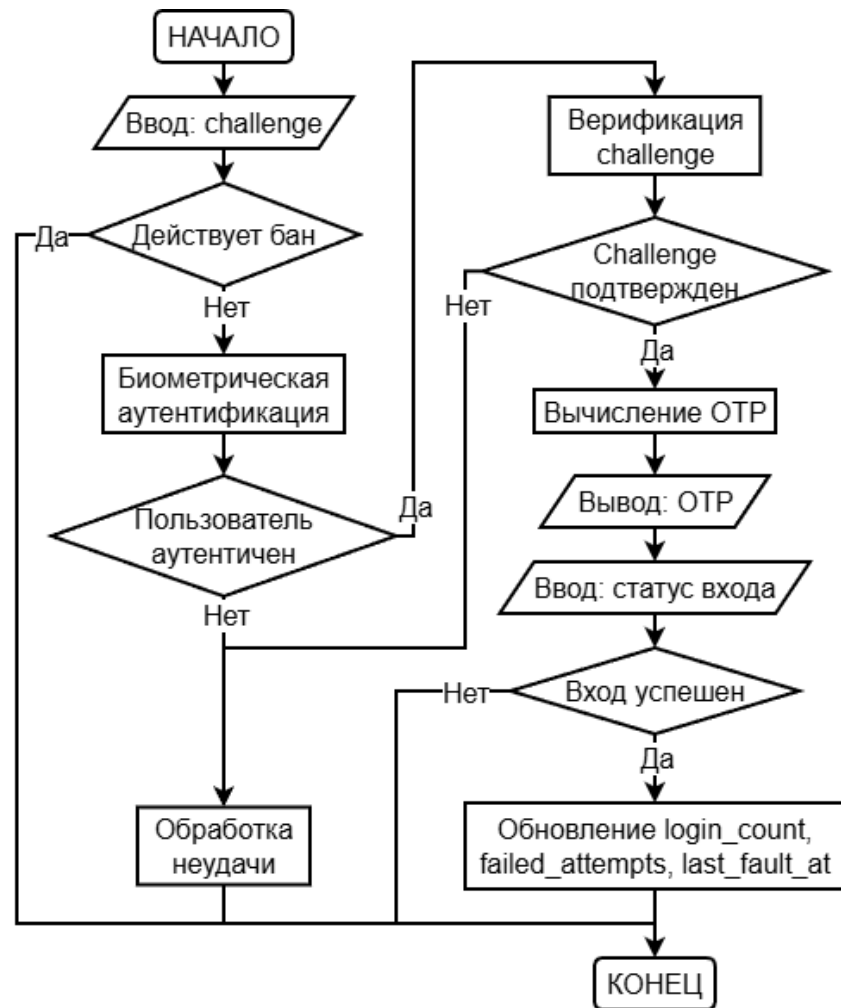


Рисунок 10 – блок-схема алгоритма регистрации OTP-токена

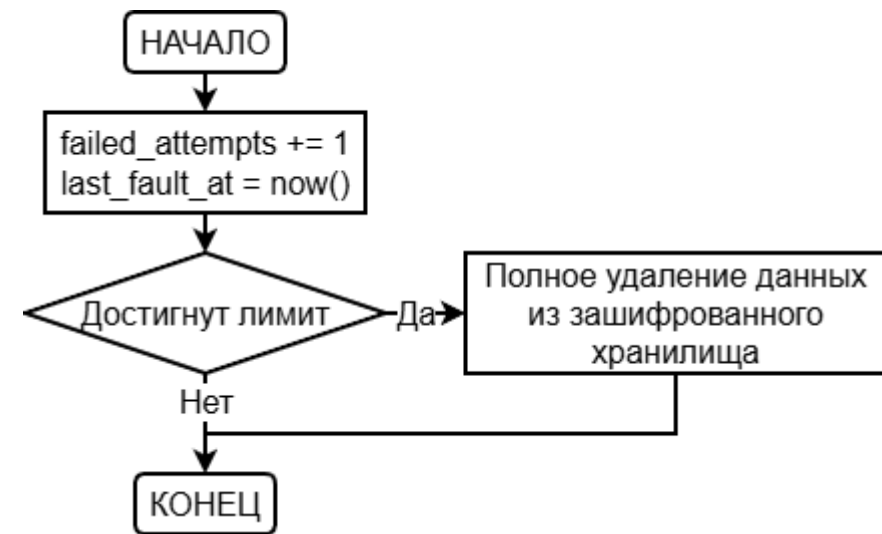


Рисунок 11 – блок-схема алгоритма обработки неудачи при активации OTP-токена

Проектирование системы: сервер

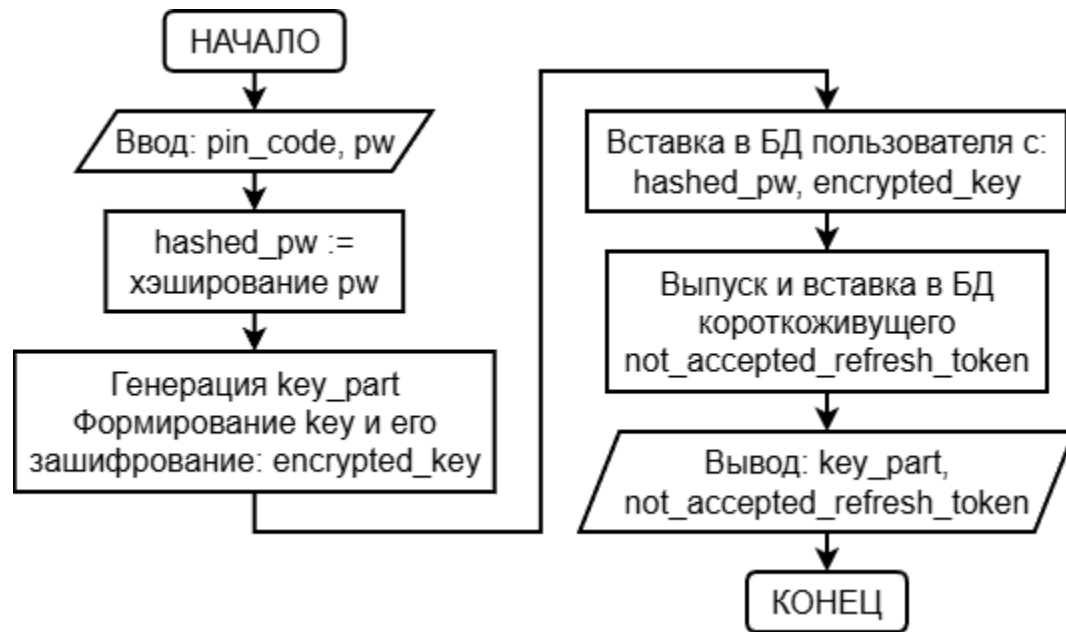


Рисунок 12 – блок-схема алгоритма инициализации регистрации пользователя на сервере

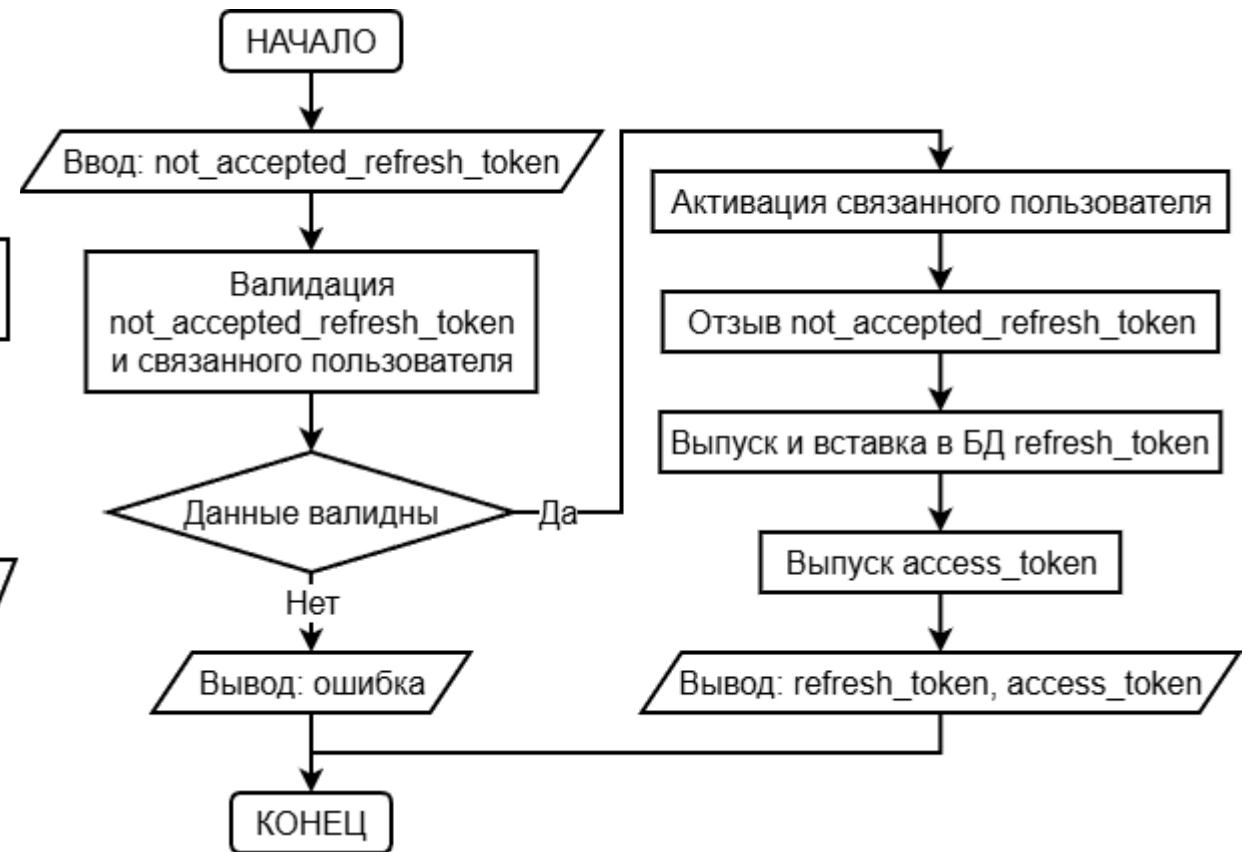


Рисунок 13 – блок-схема алгоритма подтверждения регистрации пользователя на сервере

Проектирование системы: сервер

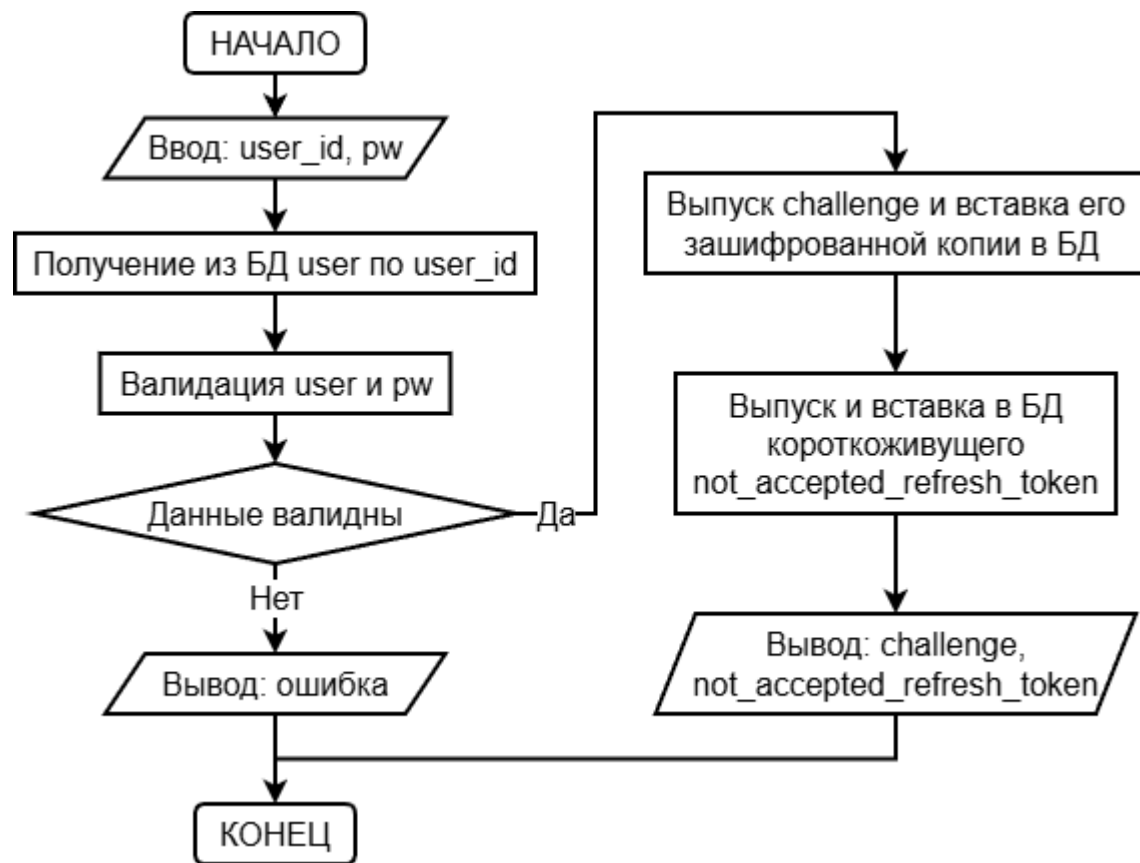


Рисунок 14 – блок-схема алгоритма инициализации входа пользователя на сервере

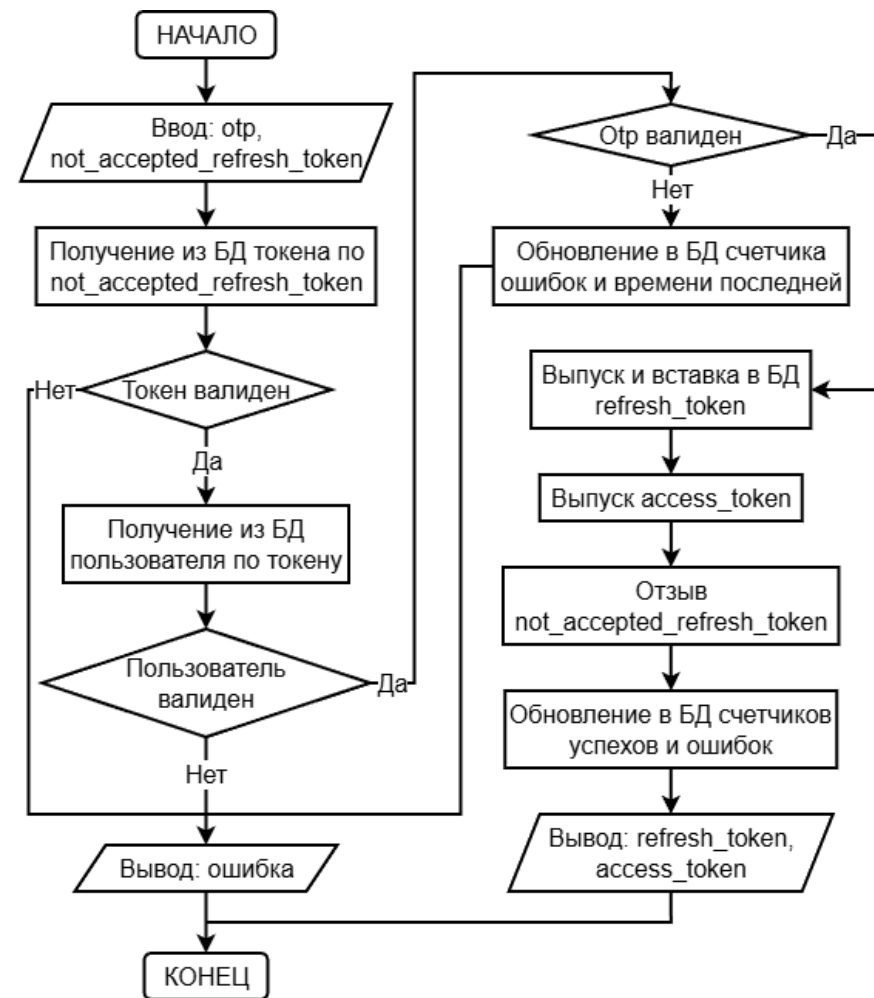


Рисунок 15 – блок-схема алгоритма подтверждения входа пользователя на сервере

Проектирование системы: сервер

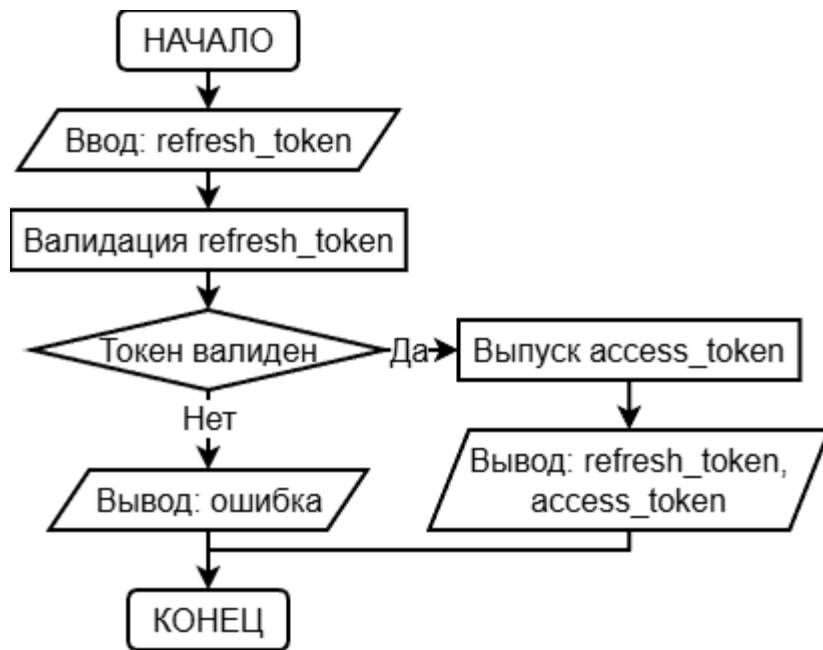


Рисунок 16 – блок-схема алгоритма обновления access-токена

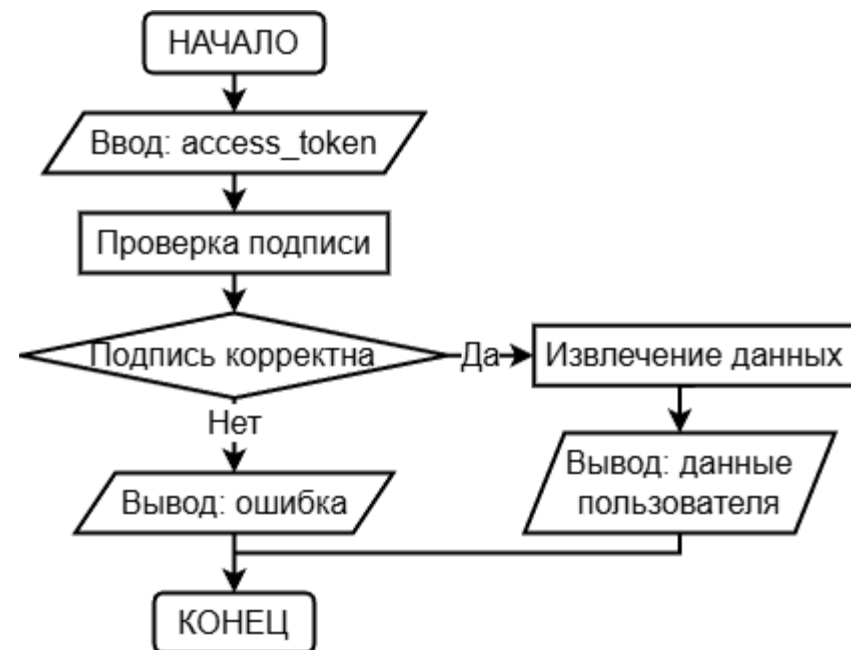


Рисунок 17 – блок-схема алгоритма верификации access-токена

Проектирование системы: сервер

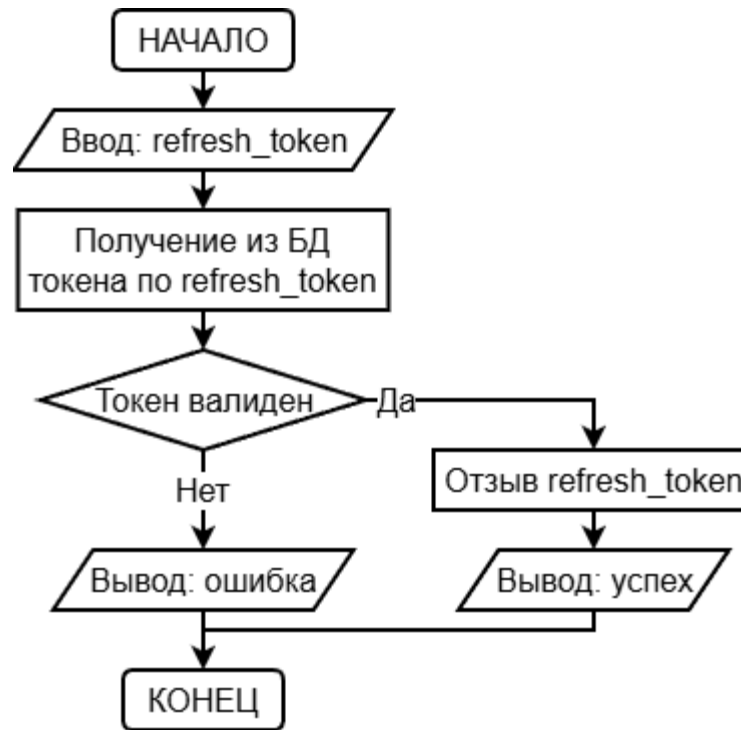


Рисунок 18 – блок-схема алгоритма выхода пользователя

Проектирование системы: технологии

Таблица 2 – Технологический стек

Компонент	Назначение	Технология	Обоснование
БД	СУБД	PostgreSQL	Надежность, поддержка ACID, масштабируемость, свободная лицензия
ОТР-токен	ЯП	Kotlin	Де-факто стандарт современной Android-разработки
Сервер	ЯП	Python	Простота разработки и поддержки, асинхронность, развитая экосистема
Сервер	Веб-API библиотека	FastAPI	Высокая производительность, асинхронность, легковесность, поддержка OpenAPI и JWT
Сервер	ORM библиотека	SQLAlchemy	Гибкость, поддержка PostgreSQL, развитая экосистема, высокая производительность, возможность исполнения низкоуровневых SQL-запросов
БД, сервер	Контейнеризация	Docker	Простота развертывания, масштабируемость, развитая экосистема

Развертывание компонентов

```
docker-compose.yml
3  services:
4    db:
5      image: postgres:17
6      container_name: auth_db
7      restart: unless-stopped
8      environment:
9        POSTGRES_USER: ${DB_USER}
10       POSTGRES_DB: ${DB_NAME}
11       POSTGRES_PASSWORD_FILE: /run/secrets/db_password
12      ports:
13        - "${DB_PORT}:${DB_PORT}" # todo УДАЛИТЬ на проде
14      secrets:
15        - db_password
16      volumes:
17        - postgres_data:/var/lib/postgresql/data
18      networks:
19        - auth_net
20      healthcheck:
21        test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
22        interval: 5s
23        timeout: 3s
24        retries: 5
25
26 > server: <9 keys>
49
50  networks:
51    auth_net:
52      driver: bridge
53
54  volumes:
55    postgres_data:
56
57  secrets:
58    db_password:
59      file: .secrets/db_password
```

Рисунок 19 – фрагмент файла docker-compose.yml

```
docker-compose.yml
3  services:
25
26 > server:
27   build: .
28   container_name: auth_server
29   restart: unless-stopped
30   env_file:
31     - .env
32   environment:
33     DB_PASSWORD_FILE: /run/secrets/db_password
34     SECRET_KEY_FILE: /run/secrets/secret_key
35     JWT_PUBLIC_KEY_FILE: /run/secrets/jwt_public_key
36     JWT_PRIVATE_KEY_FILE: /run/secrets/jwt_private_key
37   secrets:
38     - db_password
39     - secret_key
40     - jwt_public_key
41     - jwt_private_key
42   ports:
43     - "${SERVER_PORT}:${SERVER_PORT}"
44   depends_on:
45     db:
46       condition: service_healthy
47   networks:
48     - auth_net
49 > networks: auth_net
52 > volumes: postgres_data
54 secrets:
55 > db_password: <1 key>
57 secret_key:
58   file: .secrets/secret_key
59 jwt_public_key:
60   file: .secrets/jwt_public_key.pem
61 jwt_private_key:
62   file: .secrets/jwt_private_key.pem
```

Рисунок 20 – фрагмент файла docker-compose.yml

Программная реализация: сервер

```
Dockerfile x
1 >> FROM python:3.13.1-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY ./src src/
9
10 CMD ["python", "-m", "src.main"]
```

Рисунок 21 – Dockerfile сервера аутентификации

```
requirements.txt x
1 annotated-doc==0.0.4
2 annotated-types==0.7.0
3 anyio==4.13.0
4 asynpg==0.31.0
5 bcrypt==5.0.0
6 cffi==2.0.0
7 click==8.3.2
8 colorama==0.4.6
9 cryptography==46.0.7
10 fastapi==0.135.3
11 greenlet==3.3.2
12 h11==0.16.0
13 idna==3.11
14 pycparser==3.0
15 pydantic==2.12.5
16 pydantic_core==2.41.5
17 PyJWT==2.12.1
18 SQLAlchemy==2.0.49
19 starlette==1.0.0
20 typing-inspection==0.4.2
21 typing_extensions==4.15.0
22 uvicorn==0.43.0
```

Рисунок 22 – python-зависимости сервера аутентификации

Программная реализация: сервер

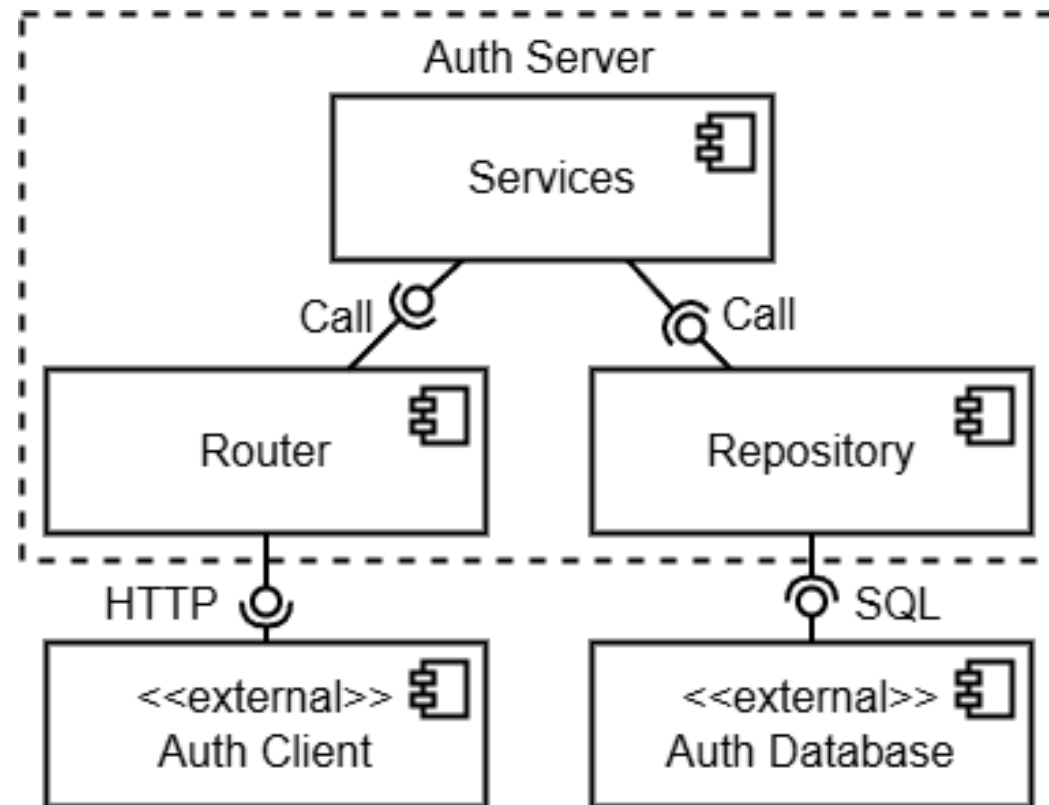


Рисунок 23 – UML-диаграмма компонентов сервера аутентификации

Программная реализация: сервер

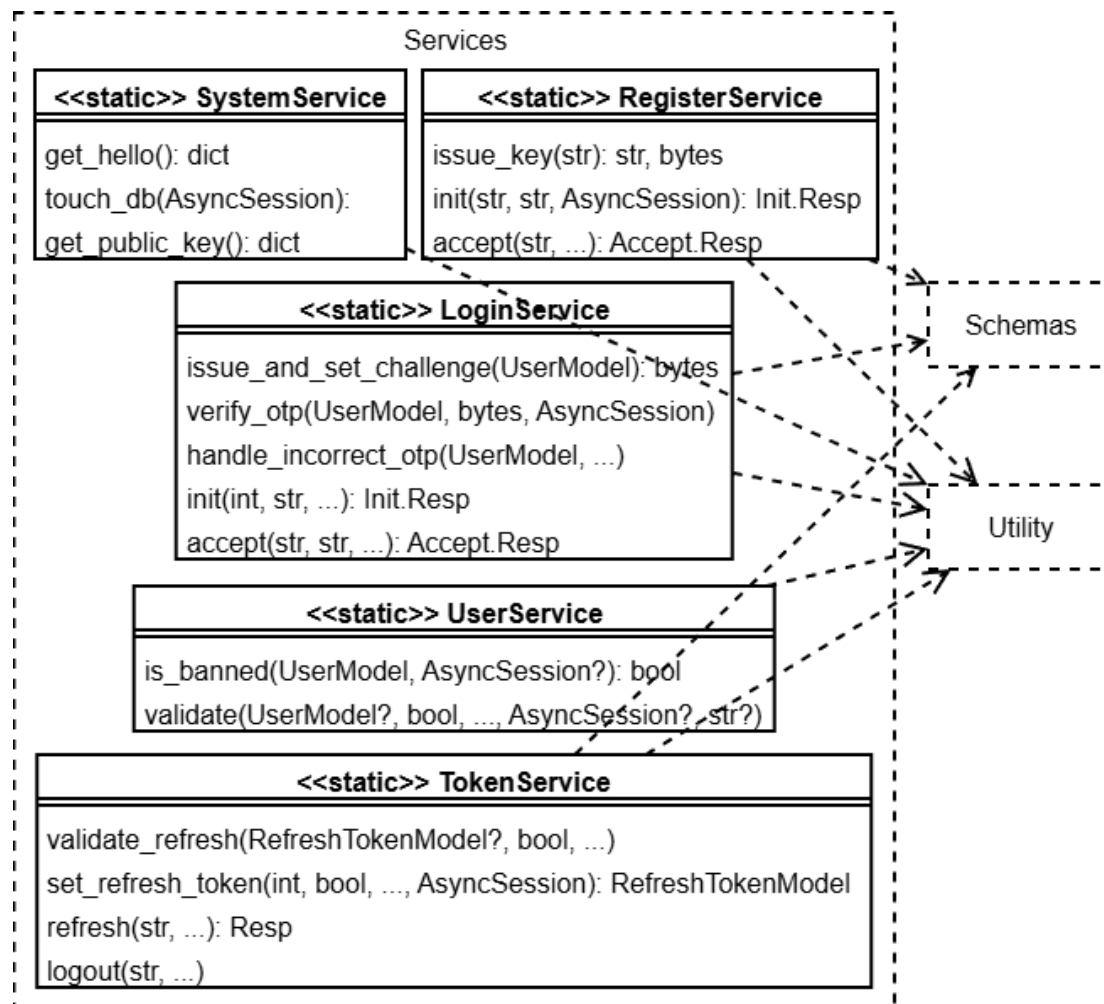


Рисунок 24 – UML-диаграмма классов сервисного слоя

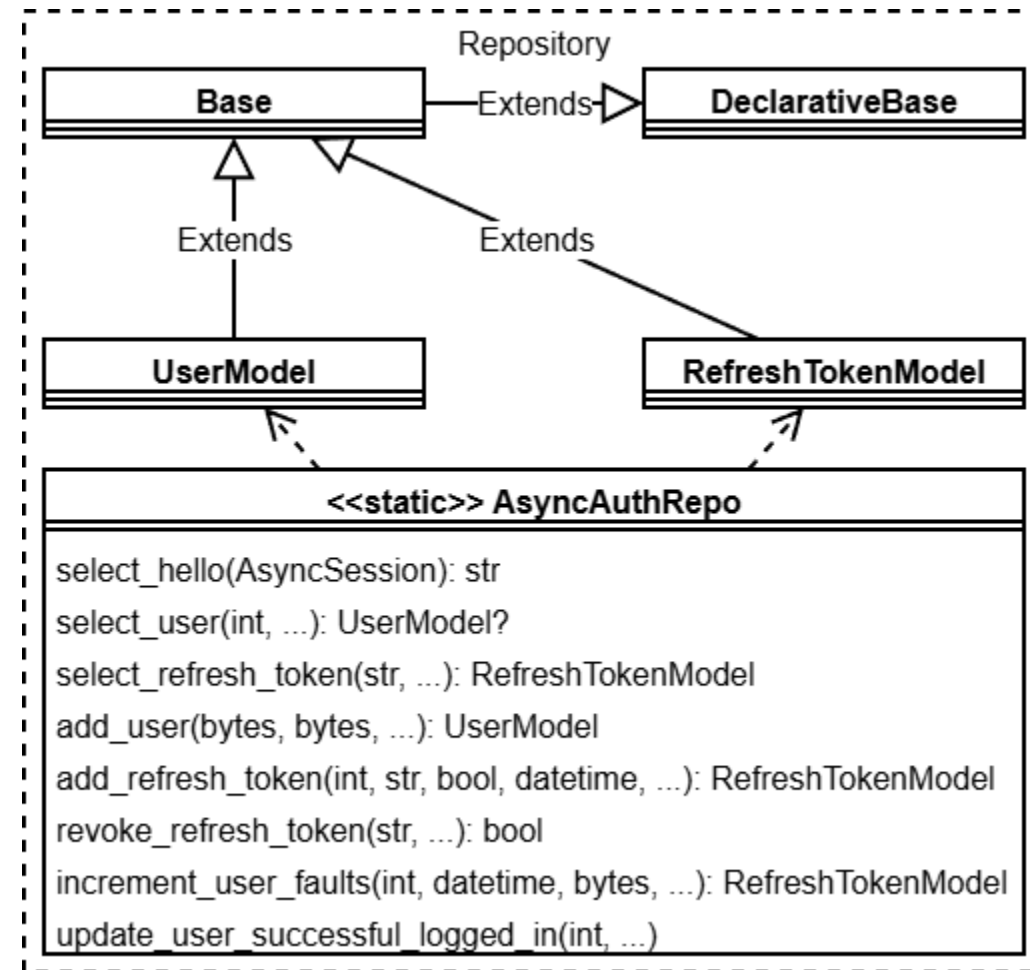


Рисунок 25 – UML-диаграмма классов слоя Repository

Программная реализация: сервер

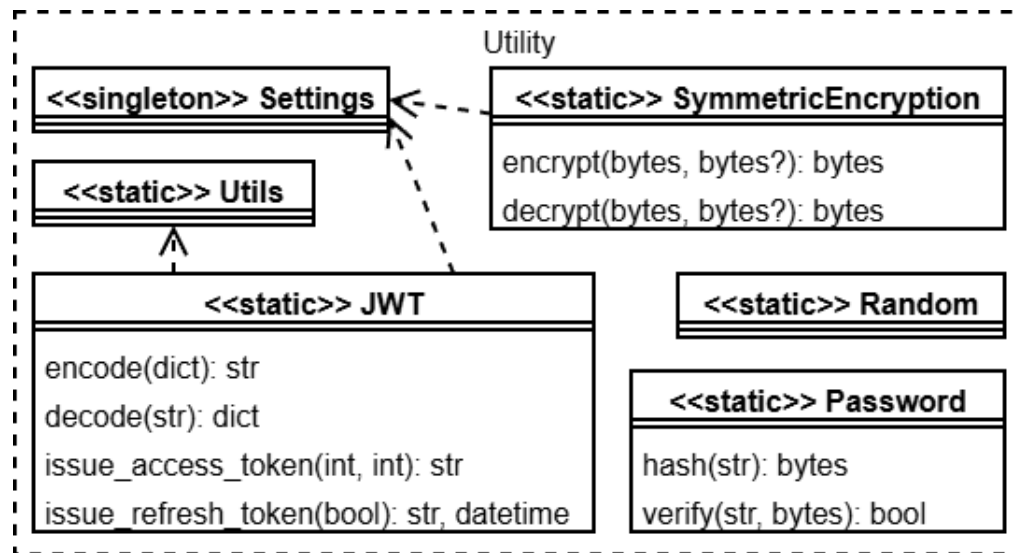


Рисунок 26 – UML-диаграмма классов слоя утилит

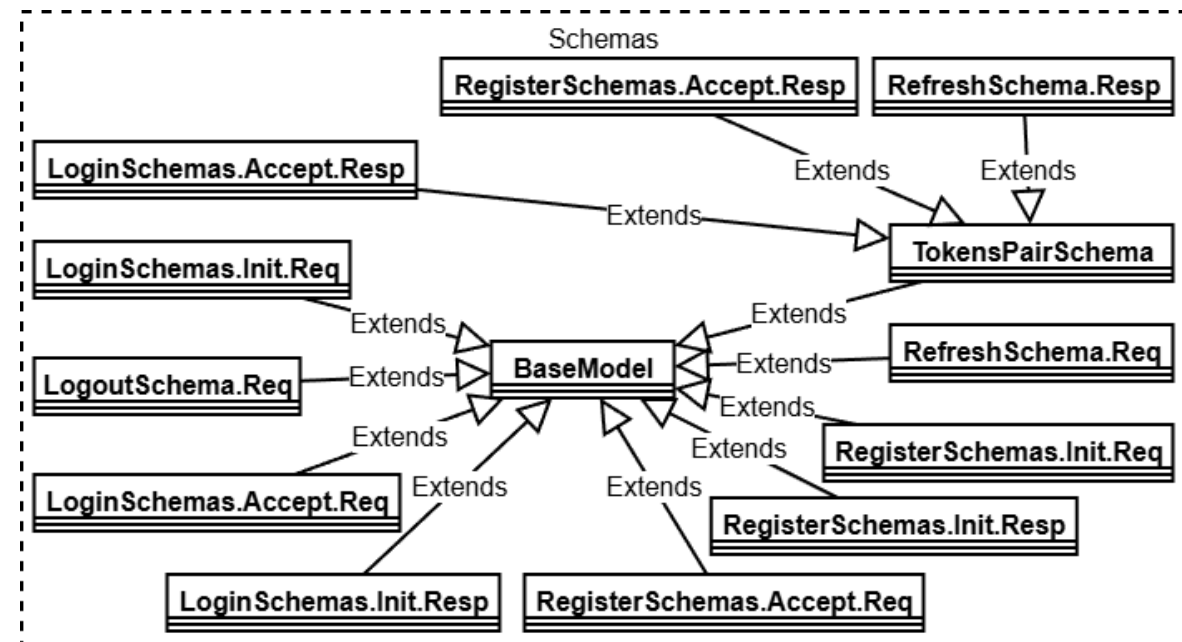


Рисунок 27 – UML-диаграмма классов слоя Pydantic-схем

Программная реализация: ОТР-токен

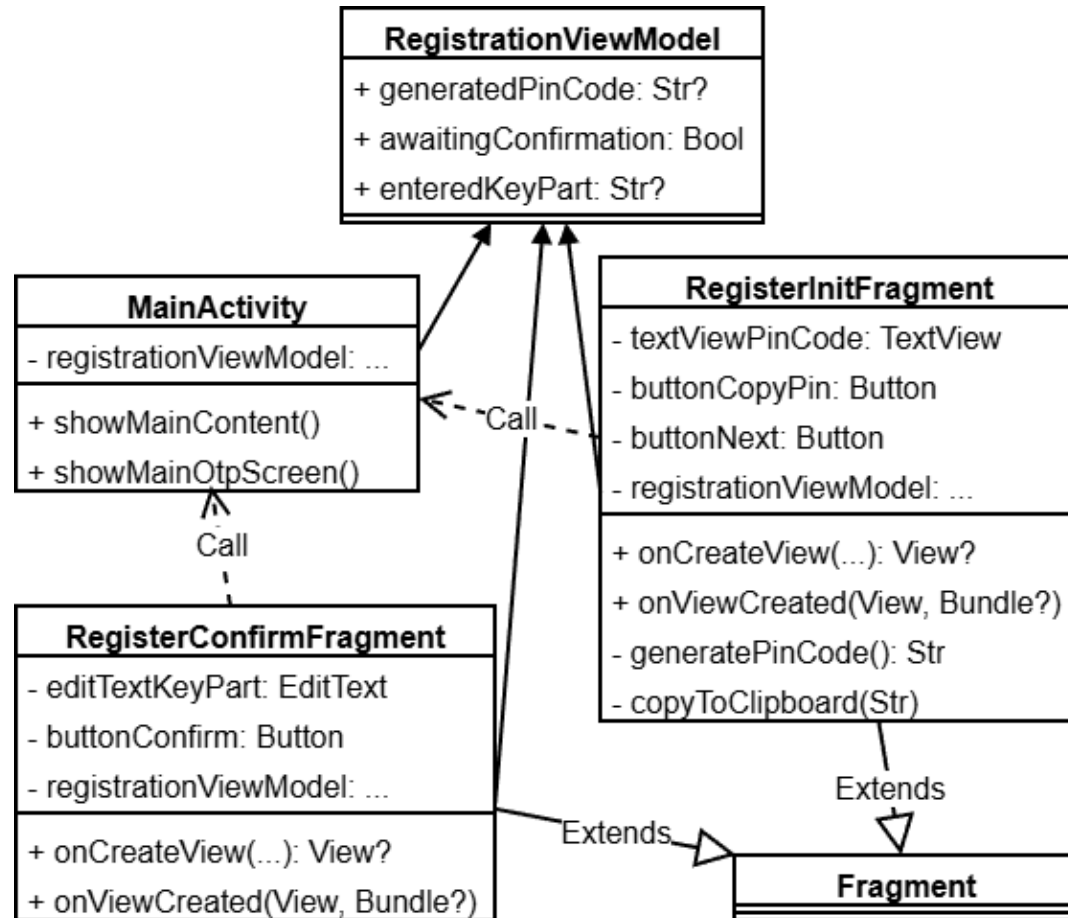


Рисунок 28 – UML-диаграмма классов регистрации ОТР-токена

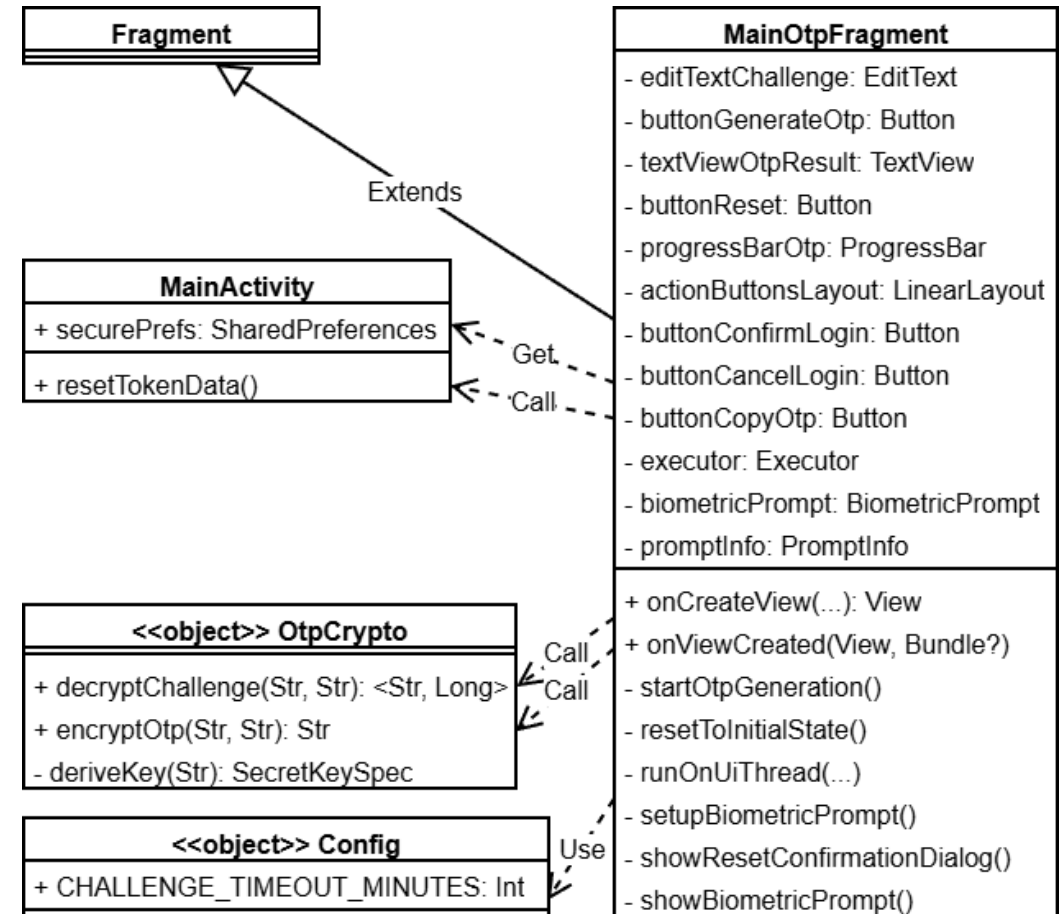


Рисунок 29 – UML-диаграмма классов активации ОТР-токена

Программная реализация: OTP-токен

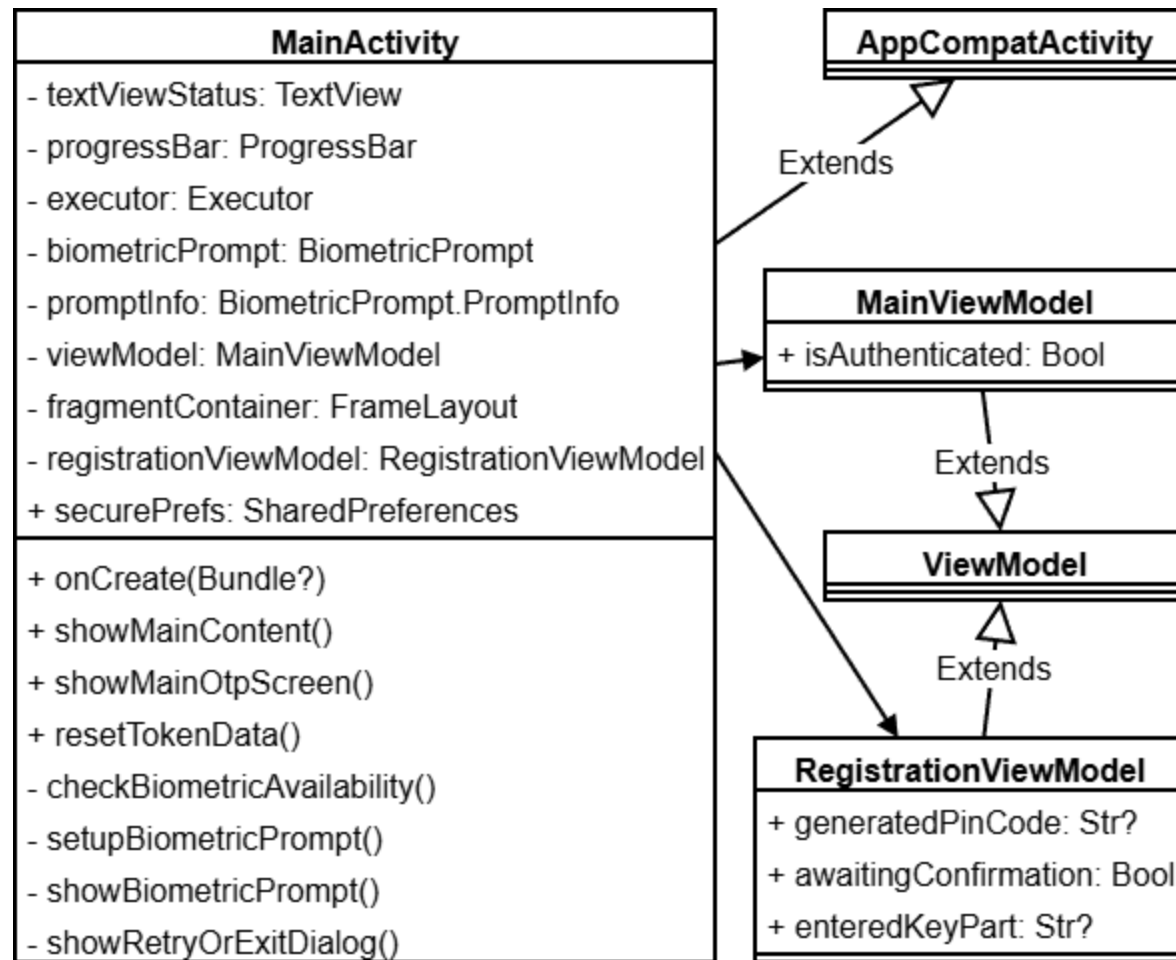


Рисунок 30 – UML-диаграмма классов точки входа OTP-токена

Виртуальная лаборатория



```
Vagrantfile
1  # -*- mode: ruby -*-
2  # vi: set ft=ruby :
3
4  Vagrant.configure("2") do |config|
5    config.vm.box = "ubuntu/jammy64"
6    config.vm.box_check_update = false
7
8    config.vm.define "ubuntu" do |ubuntu|
9      ubuntu.vm.hostname = "ubuntu"
10     ubuntu.vm.network "forwarded_port", guest: 8000, host: 8000, host_ip: "0.0.0.0"
11     ubuntu.vm.network "forwarded_port", guest: 5432, host: 5432 # todo УДАЛИТЬ на проде!
12     ubuntu.vm.provider "virtualbox" do |vb|
13       vb.name = "vagrant-vm-ubuntu-nir-spring-2026"
14     end
15     ubuntu.vm.synced_folder "../app", "/app"
16   end
17
18   config.vm.provider "virtualbox" do |vb|
19     vb.gui = false
20     vb.cpus = "4"
21     vb.memory = "8192"
22     vb.check_guest_additions=false
23   end
24
25   config.vm.provision "shell", inline: <<-SHELL
26     sudo apt-get update
27     sudo apt-get upgrade -y
28     curl -fsSL https://get.docker.com -o get-docker.sh
29     sudo sh get-docker.sh
30     sudo groupadd docker
31     sudo usermod -aG docker vagrant
32   SHELL
33 end
```

Рисунок 30 – Vagrantfile

Интеграционное тестирование

Таблица 3 – Выдержка результатов интеграционного тестирования

Название теста	Эндпоинт	Код	Время (мс)
test_warmup	/auth/touch-db	200	2020.501
test_touch_db	/auth/touch-db	200	2.679
test_register_init_success	/auth/register-init	201	216.930
test_register_accept_success	/auth/register-accept	201	37.998
test_login_init_success	/auth/login-init	200	214.525
test_refresh_success	/auth/refresh	200	33.391
test_validate_access_token_valid	/auth/validate-access-token	200	1.664
test_logout_success	/auth/logout	204	3.254

```
(.venv) PS C:\Inst\4_course\8_sem_2\NIR\server\app\tests> pytest test_api.py
-v
===== test session starts =====
platform win32 -- Python 3.13.1, pytest-9.0.3, pluggy-1.6.0 -- C:\Inst\4_course\8_sem_2\NIR\server\app\tests\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Inst\4_course\8_sem_2\NIR\server\app\tests
plugins: anyio-4.13.0
collected 17 items

test_api.py::test_warmup PASSED [ 5%]
test_api.py::test_touch_db PASSED [ 11%]
test_api.py::test_register_init_success PASSED [ 17%]
test_api.py::test_register_accept_success PASSED [ 23%]
test_api.py::test_register_accept_invalid_token PASSED [ 29%]
test_api.py::test_login_init_success PASSED [ 35%]
test_api.py::test_login_init_wrong_password PASSED [ 41%]
test_api.py::test_login_init_nonexistent_user PASSED [ 47%]
test_api.py::test_login_accept_wrong_otp PASSED [ 52%]
test_api.py::test_login_accept_without_init PASSED [ 58%]
test_api.py::test_refresh_success PASSED [ 64%]
test_api.py::test_refresh_revoked_token PASSED [ 70%]
test_api.py::test_validate_access_token_valid PASSED [ 76%]
test_api.py::test_validate_access_token_invalid PASSED [ 82%]
test_api.py::test_logout_success PASSED [ 88%]
test_api.py::test_logout_revoked PASSED [ 94%]
test_api.py::test_public_key_and_signature_verification PASSED [100%]

===== 17 passed in 5.76s =====
(.venv) PS C:\Inst\4_course\8_sem_2\NIR\server\app\tests>
```

Рисунок 31 – вывод консоли после прохождения тестов

Нагрузочное тестирование

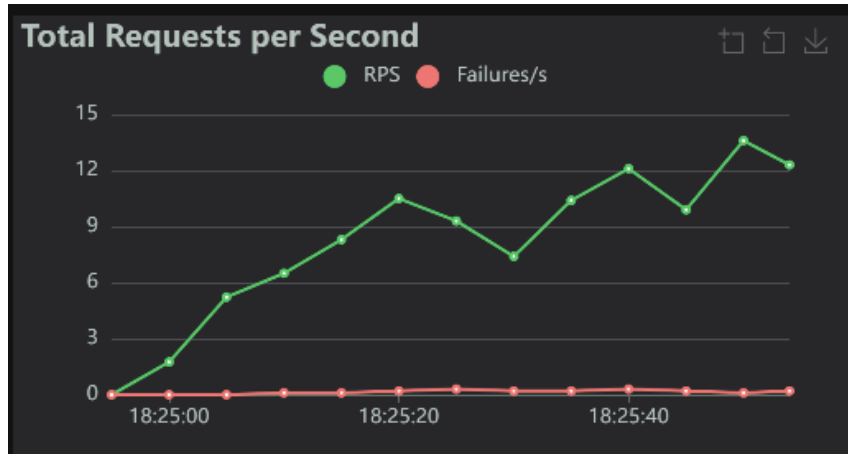


Рисунок 32 – динамика RPS

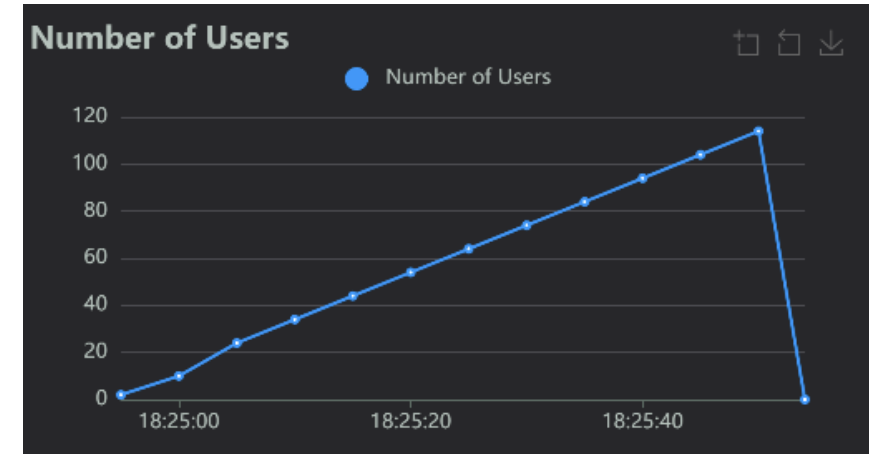


Рисунок 34 – динамика количества пользователей

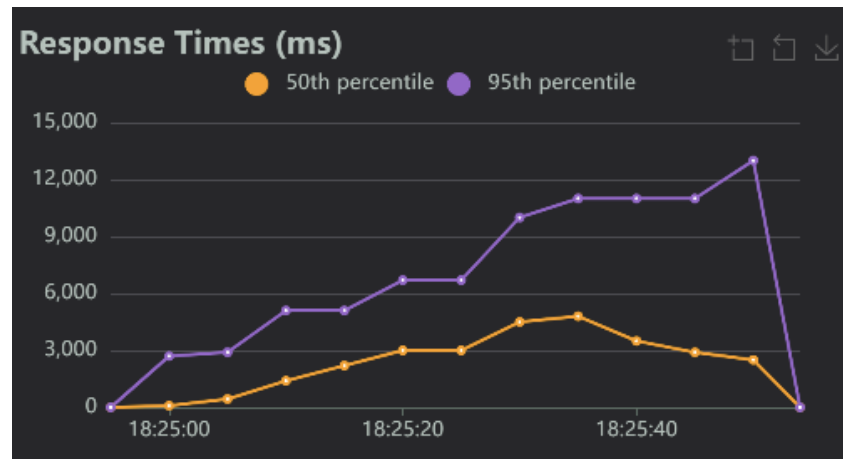


Рисунок 33 – динамика времени ответа

Комплексное тестирование

Таблица 4 – Выдержка результатов комплексного тестирования тестирования

№ сценария	Шаг сценария	Код	Время обработки запросов (мс)
1	TOTAL_SCENARIO_1	OK	1308.834
2	TOTAL_SCENARIO_2	OK	1311.647
3	TOTAL_SCENARIO_3	OK	1278.091
4	TOTAL_SCENARIO_4	OK	1276.260

```
(.venv) PS C:\Inst\4_course\8_sem_2\NIR\server\app\tests\complex> python test_complex.py
Starting complex tests (parallel)...
=== Scenario 1 started ===
=== Scenario 2 started ===
=== Scenario 3 (wrong user_id) started ===
=== Scenario 4 (wrong password) started ===
Scenario 1: waiting 70 s ...
Scenario 2: waiting 70 s ...
Scenario 4 finished.
Scenario 3 finished.
Scenario 2: waiting 60 s ...
Scenario 1 finished.
Scenario 2 finished.
Sorting results by scenario...
All scenarios done. Results in results_complex.csv
(.venv) PS C:\Inst\4_course\8_sem_2\NIR\server\app\tests\complex>
```

Рисунок 35 – вывод консоли после прохождения тестов

Заключение

В результате выполнения работы были **успешно решены** поставленные **задачи**:

- 1) **исследована предметная область**: произведен сравнительный анализ характеристик существующих популярных MFA-систем;
- 2) **смоделированы объекты и процессы предметной области**: подробно проанализированы модели систем аутентификации на основе различных факторов;
- 3) **спроектирована система мультифакторной аутентификации**: определены требования к системе, детально разработаны алгоритмы ее работы;
- 4) **выполнена программная реализация спроектированной системы**: описан процесс разворачивания и конфигурирования компонентов системы, реализованы компоненты системы, согласно принципам слоистой архитектуры и ООП;
- 5) **проведено тестирование и рефакторинг реализованной системы**: описана конфигурация виртуальной лаборатории, на которой успешно пройдены три типа тестирования: интеграционное, нагрузочное и комплексное.

Заключение

Таким образом, по итогам работы была полностью разработана и протестирована MFA-система, а значит была достигнута поставленная цель: обеспечение аутентичности пользователей информационных ресурсов вычислительной сети посредством мультифакторной аутентификации.

Практические результаты работы

Практическими результатами работы являются:

- UML-диаграммы компонентов, классов последовательностей процессов работы компонентов разработанной MFA-системы;
- BPMN-диаграмма и блок-схемы алгоритмов работы компонентов;
- ER-диаграмма базы данных аутентификации;
- сводная таблица выбранного технологического стека;
- сводные таблицы результатов проведенного тестирования системы;
- исходный код компонентов системы и модулей тестирования.

Ссылки



Публичный GitHub-репозиторий проекта



Видеодемонстрация работы системы